

10th Progress Report

W. Chan

Date:	8/04/2024
Subject:	RL for Flying-V
Supervisors:	Dr. E.J. van Kampen
Project term:	01/2024 - 08/2024
E-mail:	w.y.chan@student.tudelft.nl

Outcome of the previous meeting

- Established need to finalize Flying-V decision before the start of 2nd research phase (mid May), but I will try to reach a decision sooner.
- Discussed at length about issue of eligibility traces with neural networks, and Erik presented the idea of writing out a full matrix of derivatives for all neural network outputs. I think this is basically a neural network Jacobian (sounds like a cool name), will derive this for the critic and actor currently existing in IDHP and implement it, hopefully it works.
- Went over this new potential research objective:

Research Objective

To advance the state-of-the-art reinforcement learning based flight controllers and improve their fault tolerance, by researching novel sample efficiency improving augmentations to highly-adaptive reinforcement learning algorithms.

Description of the results since last meeting

- Fairly certain i will use the Citation's dynamics for my research instead of the Flying-V.
- Implemented eligibility traces for IDHP.

The eligibility traces are defined through the following update rules: first update $\mathbf{w}(\mathbf{t})$ by SGD with $\mathbf{E}(\mathbf{t} - 1)$, then update $\mathbf{E}(\mathbf{t})$ using $\nabla \mathbf{f}(\mathbf{x})$:

$$\mathbf{w}(t) = \mathbf{w}(t - 1) - \eta \delta \mathbf{E}(t - 1) \quad (1)$$

$$\mathbf{E}(t) = \lambda \gamma \mathbf{E}(t - 1) + \nabla \mathbf{f}(\mathbf{x}) \quad (2)$$

$$\mathbf{E}(0) = \mathbf{0}$$

Note $0 \leq \lambda < 1$

Where $\mathbf{w}$ is the network's weights, η is the learning rate, δ is the temporal difference error of the network, $\mathbf{E}$ is the eligibility trace, λ is the eligibility trace decay rate, $\mathbf{f}(\mathbf{x})$ denotes the neural network, and $\nabla \mathbf{f}(\mathbf{x})$ represents the partial derivative of the network's output wrt all its' parameters, this partial derivative will be referred to as the network Jacobian.

To derive this network Jacobian, a neural network with n outputs is treated as a system of n equations, which are parameterized by m number of weights or parameters. The network Jacobian $\nabla \mathbf{f}(\mathbf{x})$ is thus an $n \times m$ matrix, as is the eligibility trace $\mathbf{E}$. Additionally, the temporal difference error variable will also take the dimension of the network output, intuitively this can be understood by seeing that for each network output, there is a certain error which exists:

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \mathbf{f}_1(\mathbf{x}) \\ \vdots \\ \mathbf{f}_n(\mathbf{x}) \end{bmatrix} \in \mathcal{R}^n$$

$$\nabla \mathbf{f}(\mathbf{x}) = \begin{bmatrix} \frac{\partial \mathbf{f}_1(\mathbf{x})}{\partial \mathbf{w}_1} & \dots & \frac{\partial \mathbf{f}_1(\mathbf{x})}{\partial \mathbf{w}_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial \mathbf{f}_n(\mathbf{x})}{\partial \mathbf{w}_1} & \dots & \frac{\partial \mathbf{f}_n(\mathbf{x})}{\partial \mathbf{w}_m} \end{bmatrix} \in \mathcal{R}^{n \times m}$$

$$\delta \in \mathcal{R}^n$$

The eligibility trace $\mathbf{E}$ is then multiplied with the temporal difference error δ to yield the gradient of the network's error function wrt its' weights.

This idea is implemented and luckily IDHP runs this eligiblity trace without any issues, a comparison of IDHP with and without traces but with identical hyper-parameters are shown in Figure 1, where it can be seen that IDHP shows better tracking performance when using eligibility traces.

- Attempted hyper-parameter tuning. First attempt involved manually selecting and running single runs of IDHP for some hyper-parameter. This took a lot of man hours and ultimately the results were not always representative, since if i ran IDHP on the same hyper-parameters but over 15 RNG seeds, some of the runs would be much better or much worse¹. Then, i did a large number of monte-carlo runs by randomizing learning rates η (of critic and actor) and reward scaling κ , the specific steps were:

1. Randomly generate thousands of hyper-parameters configurations
2. run IDHP using each configuration over 15 differen RNG seeds
3. calculate 3 performance metrics per run; PSD error between reference and state α PSD's, episode return, convergence time (time where α error drops and remains below 0.05 degrees)
4. find the average value for these 3 metrics across 15 runs.

These values were then plotted in parallel coordinate plots to help with deciding what parameters to use, as shown in Figure 2. Note that the results for convergence time were not shown since it turns out the requirement was too strict, and only a handful would be able to meet this convergence requirement. This was not expected since the requirement was easily satisfied during manual tuning.

Hereafter, it became a little easier to decide what hyper-parameters to use, and after some manual fine tuning IDHP was ran on a faulty scenario to get an impression of it's performance under aircraft fault.

Observations from these experiments are that:

1. IDHP performance is very sensitive to hyper parameter used

¹Note that the only stochasticity in the entire python program is in generating the intial values for network weights, varying RNG seed thus simply influences these intial network weights. Weight updates and staet space simulation are deterministic.

2. When running the same hyper-parameters over many runs with different RNG seeds, it is difficult to find parameters that ensures all runs converged to a good tracking controller, i.e non which produced oscillatory control actions.
 3. The closer the hyperparameters are to producing controllers with 0 tracking error, the more likely they are to produce oscillatory controllers.
 4. Adding eligibility traces to IDHP overall led to better tracking performance, and latest experiments shows it also improves likelihood of convergence when compared to vanilla during faults (elevator dynamics reversal). See Figure 3.
- Current eligibility traces occasionally lead to network weights diverging during the fault scenario, with weights becoming infinity. An idea is to see if using replacing traces would mitigate this problem, where instead of accumulating the eligibility trace as in the eligibility trace update equation Equation 2, the update instead chooses to decay past traces or adopt latest network Jacobian depending on one condition:

$$\mathbf{E}(t) = \begin{cases} \nabla f(\mathbf{x}) & \text{if } \|\nabla f(\mathbf{x})\| > \|\mathbf{E}(t-1)\| \\ \lambda\gamma\mathbf{E}(t-1) & \text{otherwise} \end{cases} \quad (3)$$

Currently in the process of implementing this, still need to iron out bugs and run Monte Carlo runs to evaluate performance.

- Iterated on the research objective:

Research Objective

To advance the state-of-the-art reinforcement learning based flight controllers and improve their fault tolerance, by researching sample efficiency and improving augmentations to online reinforcement learning algorithms.

Research Questions

1. What reinforcement learning algorithm can yield a flight controller which is the most tolerant to faults?
 - (a) What RL algorithms are considered to be state-of-the-art?
 - (b) How is fault tolerance defined and tested in past research?
 - (c) Which algorithms have been shown to provide the best fault tolerance?
 - (d) What reference tracking performance have these algorithms shown in past research?
 - (e) What augmentations to reinforcement learning algorithms can be made and experimented with?
2. How can the identified algorithm be applied to control the PH-LAB research aircraft?
 - (a) How should the flight control system be structured?
 - (b) What are the variables defining the MDP in the case of controlling the PH-LAB?
3. How does the implemented flight controller perform during nominal flight and in the presence of faults?
 - (a) What flight scenarios should be designed to test the performance and fault tolerance of the controller?
 - (b) What is the nominal performance and degree of fault tolerance of the implemented controller?
 - (c) How well does the nominal flight performance of the proposed flight controller compare to other state-of-the-art controllers?
 - (d) How are the fault tolerance characteristics of the proposed flight controller compared to other state-of-the-art controllers?

Plans for next 2 weeks

1. Implement replacing traces.
2. Implement multi-step IDHP.
3. Perform a systematic check of IDHP performance under various combinations as shown in Table 1. Special consideration here is to see if i should be using the same hyper-parameters for all 6 algorithms? And if i should use the same hyper-parameters across the 4 flight scenarios. This is a decision i am not sure how to address.

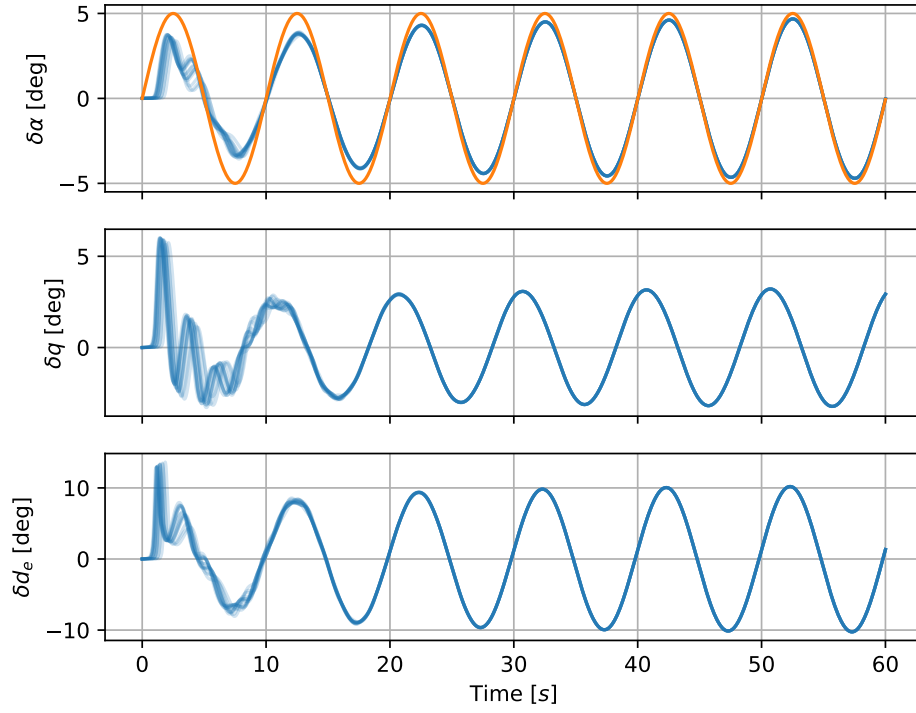
For the option of using different hyper-paramters for each combination, i might not have enough time to optimize the hyper-parameters for each combination. And it might not be a fair comparison across combinations, since the performance of IDHP might have changed because of the hyper-parameter, not because of a new flight condition or algorithm setup.

On the other hand, if i use one set of hyper-parameters for all these combinations, it might be a biased comparison to compare results across combinations as from my tests i have seen that vanilla IDHP performs well with a certain set of hyper-parameters, and if eligibility trace is added on top of this then the performance worsens; but if i optimize hyper-parameters for eligibility trace, then removing the traces results in IDHP lossing tracking performance. Furthermore, it is unclear which hyper-parameters would be used for all combinations in this

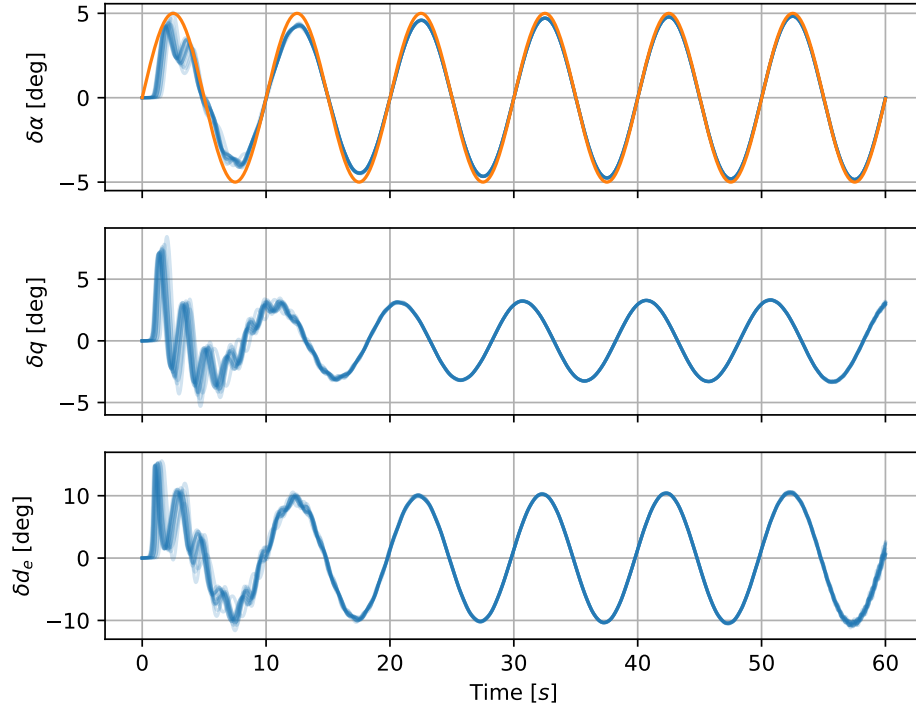
case, should these parameters be optimized for nominal flight? Or for faulty flight? Or for vanilla IDHP?

Scenario IDHP w/	nominal dynamics	elevator reversal	diminished elevator	shifted cg
Nothing	?	?	?	?
Accumulating trace	?	?	?	?
Replacing trace	?	?	?	?
Multi-step	?	?	?	?
Multi-step & accumulating trace	?	?	?	?
Multi-step & replacing trace	?	?	?	?

Table 1: Possible combinations of algorithm and flight scenarios, might be a little too much?

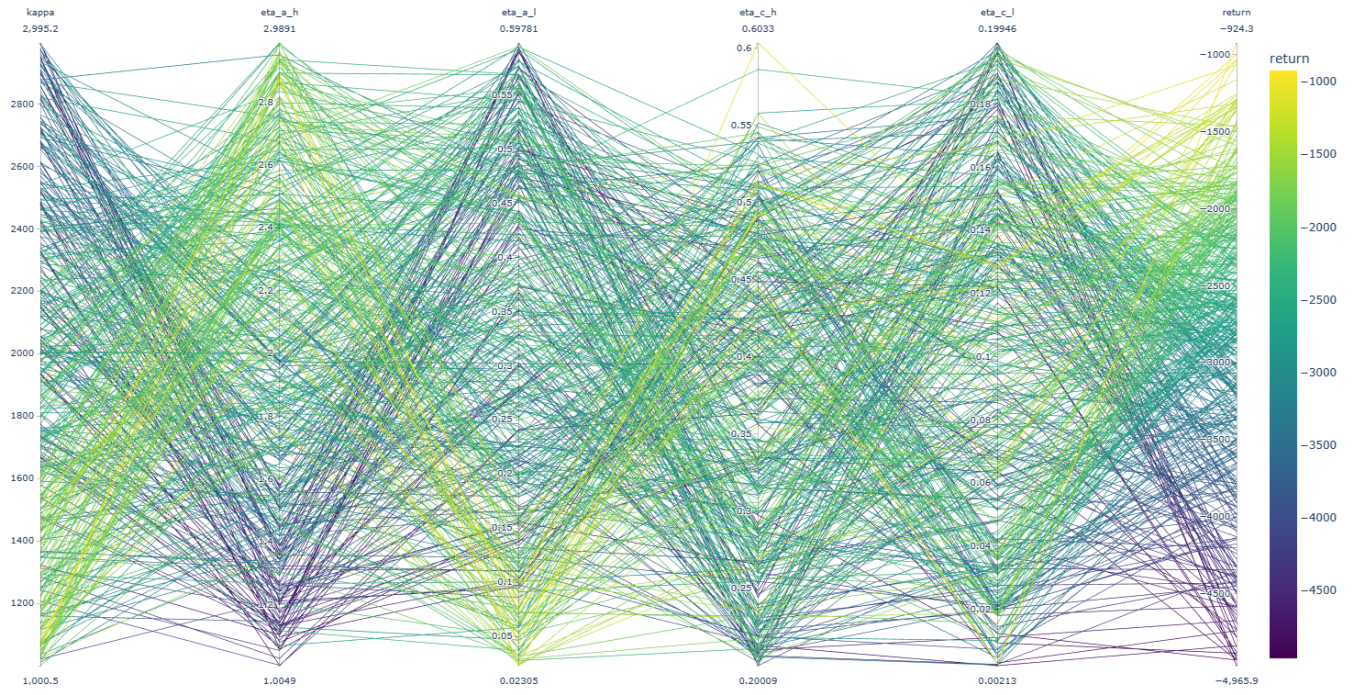


(a) Without eligibility trace, average return = -8.87, average convergence time = 33s.

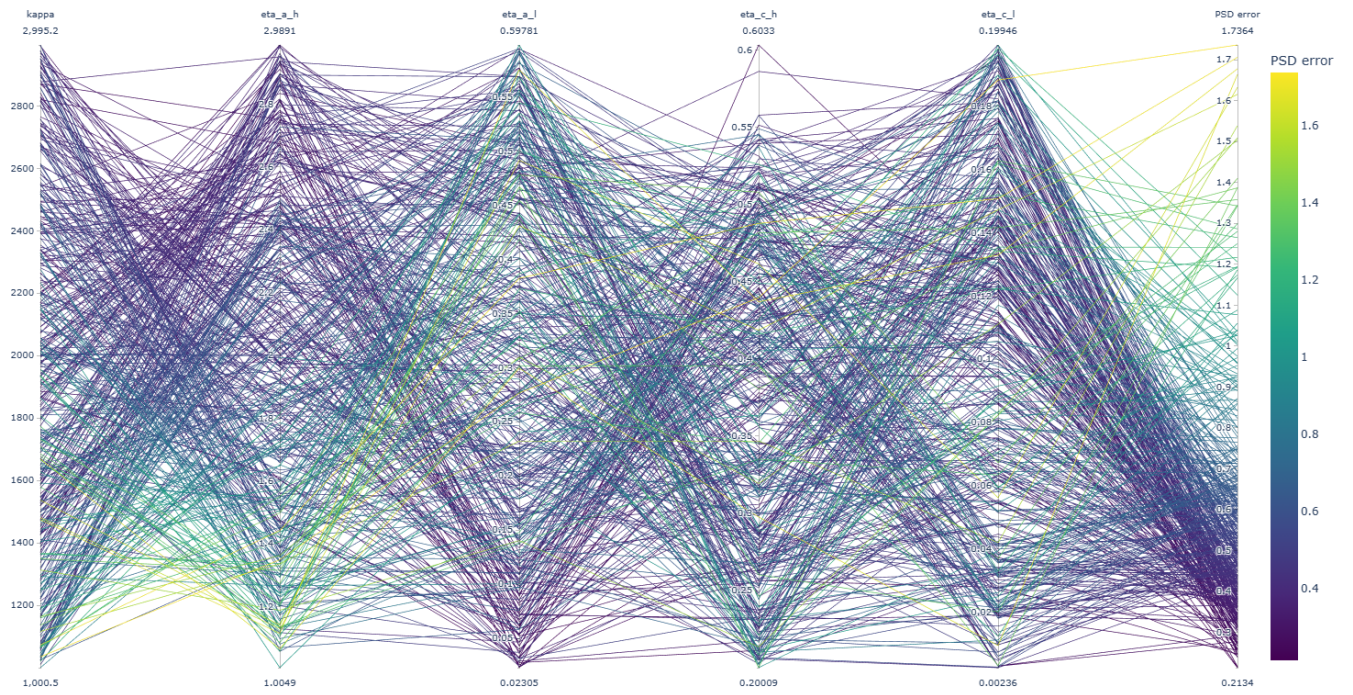


(b) With eligibility trace, average return = -4.44, average convergence time = 18 s.

Figure 1: Time trace of IDHP state and actions from 15 RNG seeds using identical hyper-parameters, convergence time defined as time where α error settles below 0.05 degrees.

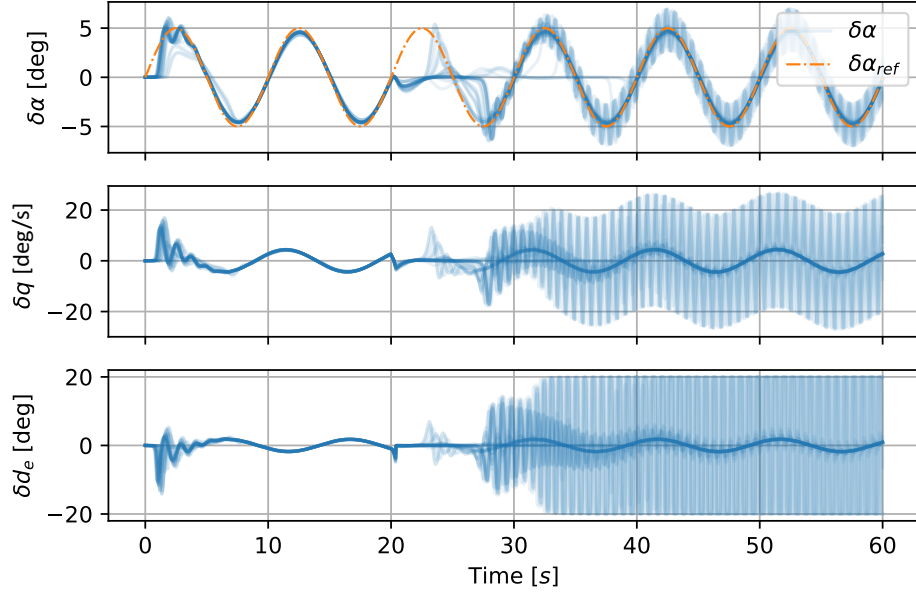


(a) Coloured according to return, yellow is better.

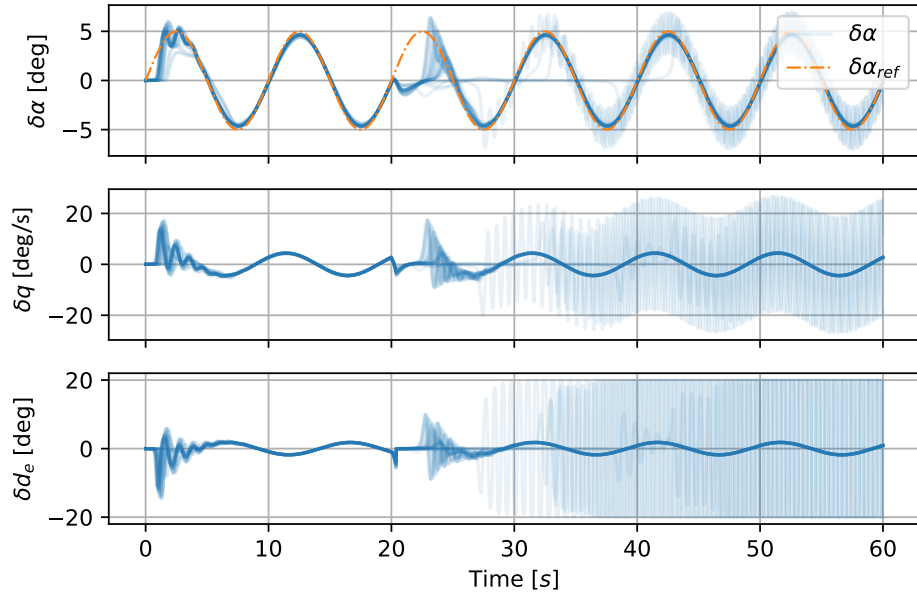


(b) Coloured according to PSD error, purple is better.

Figure 2: Parallel coordinate plots showing the return and PSD error of random combinations of hyper-parameters.



(a) Without eligibility trace, 22/30 runs failed to maintain α error below 1 deg after the 30 s mark.



(b) With eligibility trace, 11/30 runs failed to maintain α error below 1 deg after the 30 s mark.

Figure 3: Time trace of IDHP state and actions from 30 RNG seeds.